JL Engine Local

Project overview for the local-first engine, persona runtime, and operator stack

One repo, two layers:
JL Engine core + JL Platform runtime

MIT | Python 3.10+ | Windows / macOS / Linux

At a glance

Local runtime
Agent registry
Command deck UI
CLI / API access

Primary surface

/ui/ command deck
Backed by the FastAPI app

Default voice:

SparkByte

Launch local. Keep the engine local. Give the operator the full deck.

What this repository is

A local-first runtime for agent sessions, operator tools, browser control, and a rich persona system.

- JL Engine is the orchestration layer for personality, memory, cognitive state, rhythm, and backend routing.
- JL Platform is the local operator/runtime layer: API, quest sessions, interpreter, browser bridge, and workspace tools.
- The command deck, the CLI, and the API all ride on the same runtime instead of separate stacks.
- Agents are loaded as payload files through a short MPF registry, so behavior stays data-driven.
- Local tooling is powerful by design, but the trusted-machine boundary matters; the admin routes are not internet-safe by default.

Install modes

```
api | browser | ui | full
```

Entry points

```
j-engine
j-agent
jl-engine
jl-agent
cli
```

Source anchors: README.md, pyproject.toml, docs/AGENTS.md, docs/MPF_OPEN_STANDARD.md

Repository map

The codebase is split into a core engine, a platform runtime, UI surfaces, and supporting docs/tools/tests.

jl_engine_core/

Engine orchestration, memory, rhythm, cognitive gates, MPF loading, backends.

src/jl_platform/

FastAPI service, guest runtime, interpreter, browser bridge, local tools.

src/jl_engine_cli/

Headless CLI, slash commands, agent selection, backend

ui_web/ + ui_easy/

The main command deck and the lighter flow deck served by the same API.

docs/ + tools/ + tests/

Docs, generators, local automation scripts, and the regression suite.

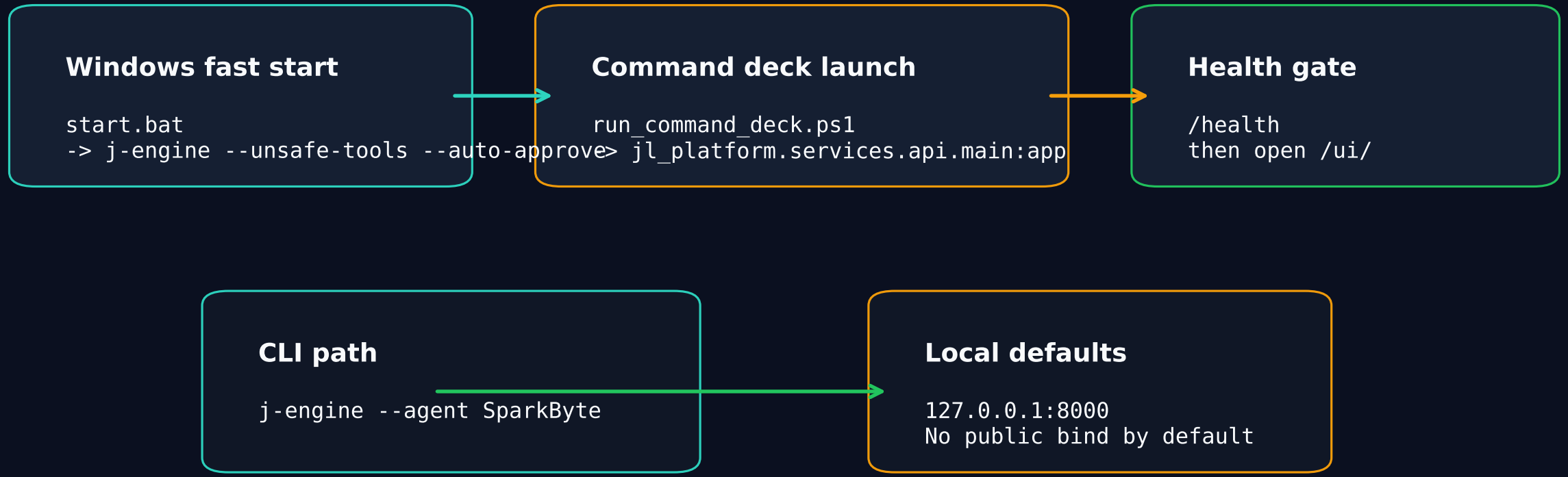
Canonical runtime data

jl_engine_core/data/agents/JL_Agents.mpf.json + payload files under jl_engine_core/data/ are the main source of truth for agent behavior.

The runtime favors data files over hard-coded personas, which makes the system easy to extend.

Launch paths

This repo has a few front doors, but they all converge on the same local runtime.



- The Windows launcher keeps the runtime local and opens the deck automatically when the health check passes.
- The CLI is separate, but it still uses the same core registry and backend configuration.

JL Engine core

This is the brain: state, agents, memory, rhythm, emotional aperture, and backend routing.

Orchestrator

JLEngineCore coordinates response generation, telemetry, and feedback.

State engines

Behavior and cognitive modes, cognitive gears, rhythm, and emotional aperture.

Memory + backends

Hybrid memory, drift pressure, and backend routing for Ollama, OpenAI,

MPF registry loader

Loads the short registry and resolves the full payload for the active agent.

Output contract

Returns reply, telemetry, and feedback so callers can show state and trace details.

Key modules

`behavior_engine.py` | `cognitive_gears.py` | `cognitive_modes.py` | `emotional_aperture.py` | `rhythm.py` | `drift_pressure.py` | `hybrid_memory`

The core is UI-agnostic, so the same engine can back the CLI, API, or future surfaces.

Platform runtime

JL Platform wraps the core engine in local workflows and admin surfaces.

Quest runtime

FatQuestRuntime manages agent sessions, side quests, clone-on-failure, and agent lineage.

Interpreter

InterpreterSession handles tool calls, direct action fallback, and confirm

Browser bridge

Playwright-backed local browser control and inspection.

Workspace tools

List, read, save, and review files inside the workspace root.

Self-edit loop

A protected local loop for controlled self-modify

API surface

`/quest/* | /interpreter/run | /browser/* | /workspace/* | /self-edit/* | /tools/*`

The security model assumes this runs on a trusted local machine unless you add your own boundary.

Agents and MPF

Agents are data, not hard-coded behavior. The registry points to payloads and the payloads carry the persona.

How it works

1. Read the MPF registry entry.
2. Resolve the payload file.
3. Expand modular payloads if needed.
4. Load the resolved persona into the active session.

Built-in agent examples

SparkByte | Slappy | The Gremlin | Supervisor
SaaS Copywriter | Cold Outreach Assistant
YouTube Scriptwriter | Brand Voice Generator
Startup Pitch Writer | Forgebinder

Registry path

`jl_engine_core/data/agents/JL_Agents.mpf.json`

Payload families

`fat_agents/` | `jl_agents/` | `generated/`

The registry stays small; the payloads carry the full behavior, voice, and backend defaults.

UI surfaces and user flows

The deck is the main product surface, but the CLI and the lighter UI are part of the same system.

/ui/

Full command deck
Chat, agents, tools, files, browser, self-edit

/ui-easy/

Lighter flow deck
Reduced surface area

CLI

j-engine
Slash commands
Agent switching
Backend selection

- The web UI is not a separate app; it is a view over the same API and runtime state.
- Agent selection, workspace browsing, and tool execution all stay inside the command deck flow.
- The UI also reflects browser-session state, benchmark state, and self-edit status.

UI behavior is driven by ui_web/app.js and backed by the same local API routes.

Tools, tests, and safety

This repo is powerful by design, so the trust boundary and regression coverage matter.

Local operator surfaces

shell, cc bridge, browser, workspace, forge, audit, self-edit, and privileged remote tools.

Safer defaults

Keep the service on 127.0.0.1, gate dangerous actions, and treat admin

Tests

pytest coverage for runtime, browser bridge, tool forge, memory, CLI, schedu

Bench manager

tools/bench_manager.py w

The main safety move is simple: do not expose the local admin surfaces to the public internet without adding auth and proxy controls.

Docs and next steps

The docs are unusually complete. Start there, then use the deck or CLI to exercise the runtime.

Read first

README.md
ONBOARDING.md
ARCHITECTURE.md

Runtime docs

docs/AGENTS.md
docs/MPF_OPEN_STANDARD.md
docs/TOOL_FORGE.md

Safety docs

SECURITY.md
TROUBLESHOOTING.md
docs/ERROR_HANDLING.md

- Launch locally, pick SparkByte, and verify the health endpoint before touching the heavier surfaces.
- Treat the operator tools as local-only until you explicitly harden them for remote access.
- If the project changes, regenerate the deck with the script so the overview stays current.

Generated from current checkout

JL_Engine-local-re-skin1 (1) | 2026-03-18 17:16